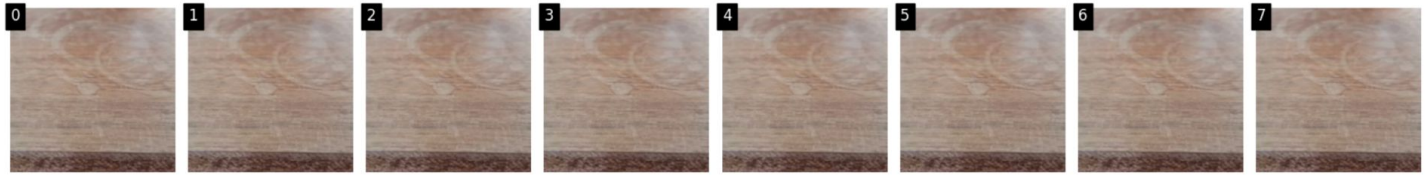
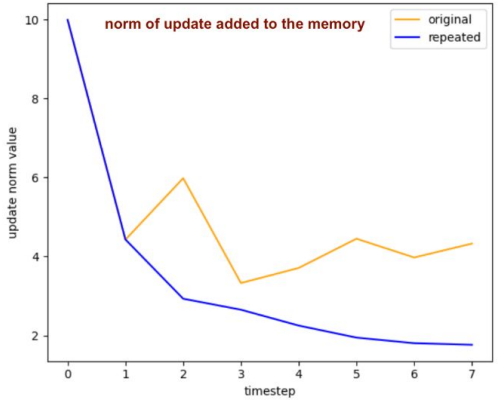
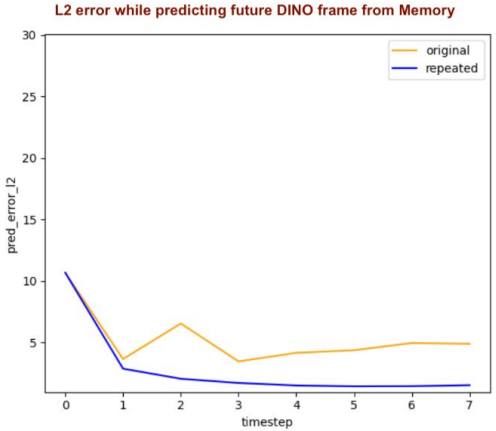
original video

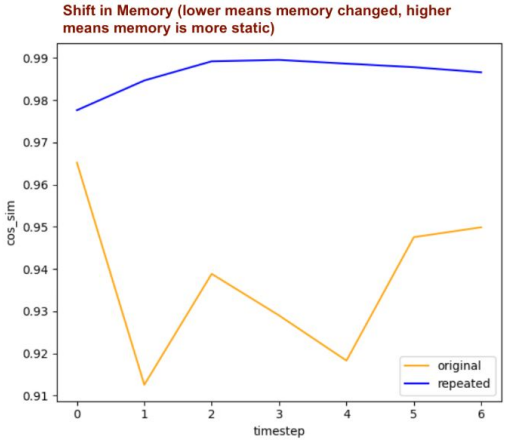
repeated video

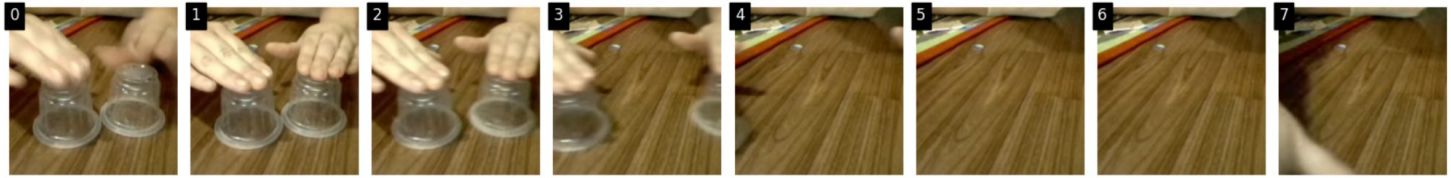


GT: Putting something upright on the table
Pred with original frames: Putting something on a surface
Pred with repeated frame: Lifting something with something on it



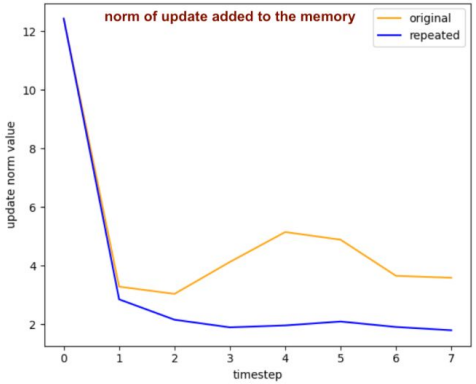


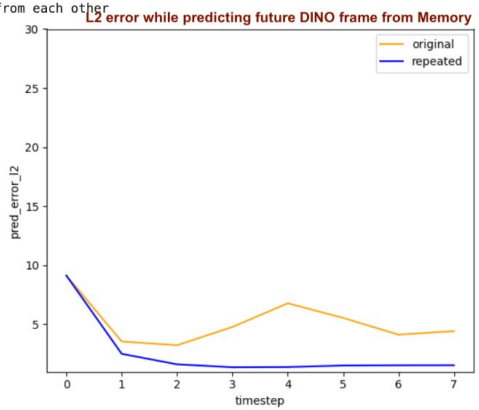


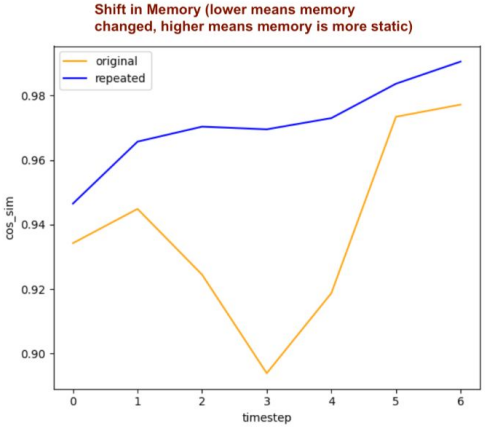


repeated video

GT: Moving something and something away from each other
Pred with original frames: Moving something and something away from each other
Pred with repeated frame: Moving something and something away from each other







Towards General Purpose Robots

Manu Gaur

Long-Horizon Robotics is Memory-Bottlenecked

Without an abstraction of the past, it is impossible to form a causal understanding of how the world evolves and to model its dynamics.

- As the individual skills become more robust, the bottleneck is increasingly in how the robot deploys these skills to solve complex tasks, not the skills themselves.
- **Complex tasks need narrative memory**. "Making coffee" means tracking task stage, object locations (even out of view), and what's already been tried (to avoid repeating mistakes)
- **Attention alone can't scale.** Keeping a full observation history over minutes is infeasible, but discarding it leads to incoherent behavior.

The model needs to keep track of time to cook the grilled cheese

The model needs to remember its progress wiping the window

Why Future Prediction?

Predicting the future forces the model to learn:

- operate at a higher **temporal abstraction** – what task stage am I in, and what comes next?

- **Dynamics** – how objects move, interact, and change state

- a ***persistent memory*** to deal with **partial observability** – inferring what's out of view from what was seen before

This is the same principle the brain uses – prediction error drives attention, memory and learning. [Friston, 2010]

1. Karl Friston. The free-energy principle: a unified brain theory? Nature reviews neuroscience, 2010.

Problem Formulation

Future prediction can:
- operate at higher temporal abstraction
- encode task stage → better for multimodal futures
- deal with partial observability

LLM can do all the above; Q: how about spatial causality (MPC)?

Problems with training future predictor:
- mode collapse with MSE
  - generative or contrastive?
- compounding errors: curriculum over horizon lengths (can do with arch as well)
- shortcut learning

Long horizon BPTT not feasible:
GPU and Data constrained (RL has data, but unstable over long horizons)
- stage 1: linear TTT over chunks
- stage 2: Encoder-decoder autoregressive over long horizons.
- stage 3: downstream transfer (policy head and VQA head)

Pixels vs Latents?

Planning horizon
- low level policy: reactive, CroBo
- MPC/MBRL: forward dynamics over long horizons = better planning

Zero-shot Planning

Table 1. Planning results for offline world models on six control environments.

Model	Maze SR ↑	Wall SR ↑	Reach SR ↑	PushT SR ↑	Rope CD ↓	Granular CD ↓
IRIS	0.74	0.04	0.18	0.32	1.11	0.37
DreamerV3	**1.00**	**1.00**	0.64	0.30	2.49	1.05
TD-MPC2	0.00	0.00	0.00	0.00	2.52	1.21
Ours	0.98	0.96	**0.92**	**0.90**	**0.41**	**0.26**

Data
- if not using VQA and fully self-supervised
  - stage 1 : standard shot vid datasets
  - stage 2: ego4D; hour long videos
  - stage 3: tasks specific environments

Chunk compression
- *State*:
  - TTT (recurrence)
  - Dynamic token dropping (causal autoregressive)
- *Losses*: Future prediction, Scene Flow, VQA?

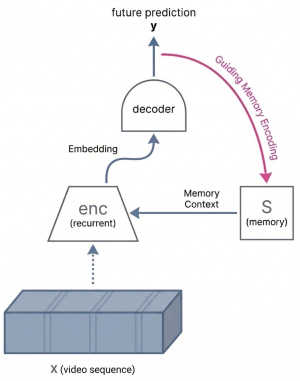
Defining a chunk
- not need if doing token dropping
- for stateful training
  - stage 1: video = chunk
  - stage 2: fixed K frames
    - can heuristics with thresholds→ covisibility/future pred
  - stage 3, task stage = chunk

Ambitious: Meta learnt

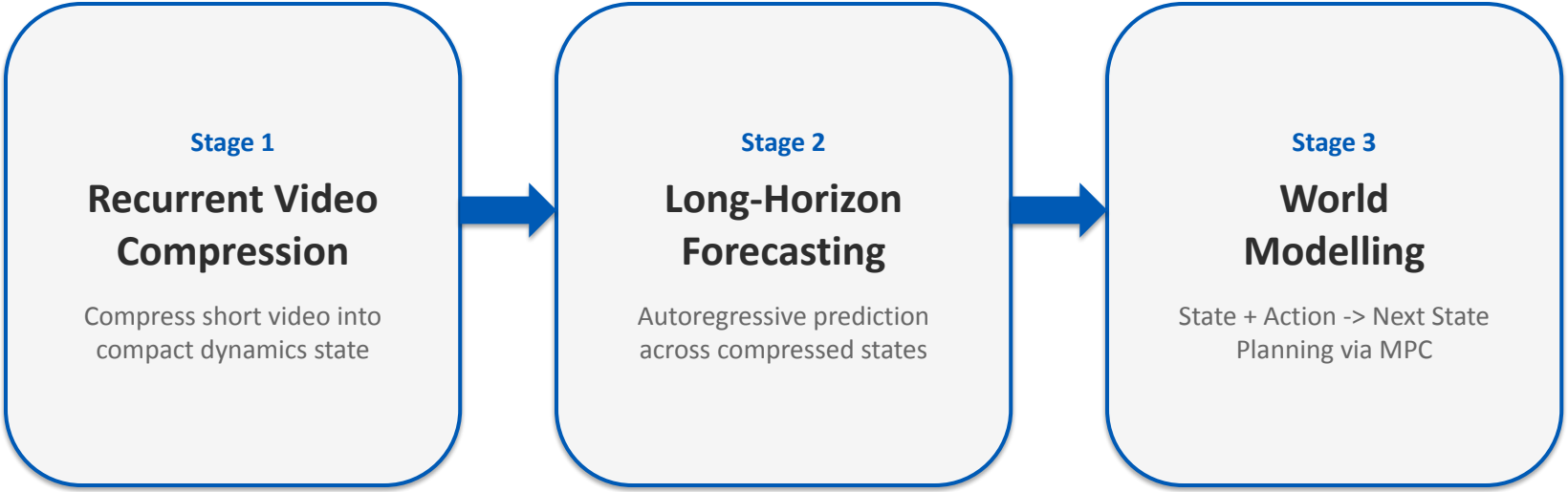
Post-training (the fun part!! 😍)
- RMA: dense supervision; privileged info = KV cache
- Quiet-Star, SEAL etc. (repurpose CL/reasoning tasks as memory management)

Evaluation
- Long horizon sims
- Ego4D
- EPM env → object retrieval/navigation

Baselines
- CroBo, ToBo
- NE-Dreamer



Three-Stage Approach



Long-horizon BPTT is infeasible (GPU + data). Staged approach makes it tractable.

Stage 1: Learning Temporal Dynamics

Goal: compress video into a state expressive enough for stage 2 forecasting

Architecture:

Frozen DINO ViT-S/14 backbone (384-dim features); SSv2, 8 uniformly sampled frames

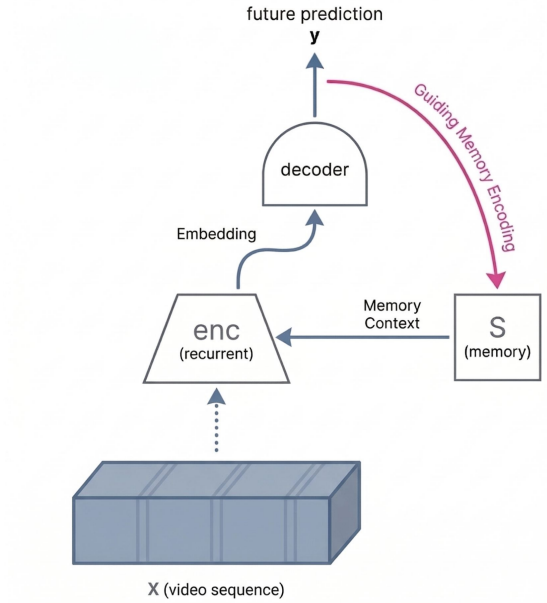
Additive RNN with predictive coding update:

```
Error = DINO(x_t) - W_pred(S_{t-1})
S_t = LN(S_{t-1} + W(Error))
```

- Bounded memory footprint. We simply cross-attend to the latent memory

Training:

- Self-supervised: predict future DINO features
- Auxiliary loss: scene flow for more expressive state



The compressed state encodes environment dynamics: *Cars decelerate. Pedestrians change direction. Occlusions resolve*

Stage 2: Long Horizon Forecasting

From Reaction to Anticipation: *A 1km construction zone. Reactive systems brake hard at the cones. A long-horizon model sees the signs 30 seconds earlier, smoothly decelerates, and finds a merge gap*

How:

- Stage 1 encoder compresses video chunks into compact states
- Encoder-Decoder Autoregressive transformer predicts across chunks (parallelizable training)
- Long-horizon BPTT is infeasible → this staged approach makes it tractable

Encoded representation understands compositional structure of tasks: "highway merge" = check-mirror → signal → gap-find → accelerate → merge → stabilize.

Such representation would yield sample-efficient and robust policies.

Stage 3: World Modelling

The leap: State + Action → Next State. Now, the model can "imagine" consequences of steering/braking decisions before executing them.

- **Planning**: MPC / tree-search over imagined futures
- **Policy training**: Train policies inside the world model (no real-world rollouts needed)
- **Simulation at scale**: The model IS the simulator — learned from data, not engineered

The Full Stack for General-Purpose Robotics:

```
LLM:  "Merge left, take the exit in 800m"
  ↓
WM:   "Accelerate to 120, truck clears in 3s — safe gap"
  ↓
 π:   Steering + throttle at 20Hz
```

The LLM knows what to do. The world model figures out how. The policy does it.

Models Trained

#	Model	Prediction Target	Loss	Status
1	zyvsy8gk	next frame, learned space h=W(DINO_cls)	CE + 0.1*L2	works
2	--	next frame, learned space, pred only	L2 only	collapsed
3a	2ldiw9xk	next CLS in DINO space	L2	works
3b	e6esmgmu	next patches in DINO space	L2	works

Takeaway: prediction-only in learned space collapses;

It doesn't collapse with action-classification;

Q: is the recurrent state learning **general dynamics** or **overfitting** to SSV2 + action recognition?

Transfer Probe: UCF101

Does the recurrent state generalize beyond SSv2?

Freeze everything, linear head on UCF101 (101 classes, 13K videos, 20 epochs)

Model	UCF101 Acc
DINO mean-pool (no temporal modeling)	88.0%
DINO concat (8x384=3072-dim)	86.0%
RNN CLS, DINO-space pred	85.4%
RNN CLS, CE+pred, learned space	84.0%
RNN Patches, DINO-space pred	81.7%
RNN Patches, CE+pred, learned space	78.3%

1. All RNN states trail DINO mean-pool (88%) || CLS > patches → maybe these change with better *W_pred*
2. DINO-space > learned space -- action recognition suboptimal (scene flow, sigreg,ema)
3. mean-pool > concat suggests UCF measures appearance, not temporal reasoning

Dino-space Prediction: How Lossy is Recurrence?

Causal transformer = full attention over past frames (no state bottleneck). Upper bound for recurrence.

CLS token (S=1, D=384)

Model	Pred L2
Copy current frame	609
Causal Transformer	517
RNN	513

Patch tokens (S=256, D=384)

Model	Pred L2
Copy current frame	1085
Causal Transformer	783
RNN	851

RNN = causal transformer ⟶ recurrent state not a bottleneck

Both models beat copy-frame baseline

RNN trails transformer by ~8% -- need more expressive W_pred, maybe better state update

attention can explicitly model per-patch motion across frames

Temporal Shuffle Test

Model beats copy baseline by 17-24% -- is that from learning dynamics or just averaging recent frames?

Shuffling frame order tests this: dynamics break under shuffling, averaging doesn't.

Ratio = shuffled_loss / normal_loss (higher = more temporal dependence)

Model	Normal	Shuffled	Ratio	Copy Baseline
CE+pred, CLS, learned space	2.23	2.29	1.03x	--
CE+pred, patches, learned space	3.80	4.29	1.13x	--
Pred only, CLS, DINO space	176.9	211.9	1.20x	620
Pred only, patches, DINO space	851.5	1114.1	1.31x	1116

Copy Baseline Shuffle Sensitivity

	Copy Shuffle Ratio (data)	Model Shuffle Ratio
CLS (DINO space)	11.2x	1.20x
Patches (DINO space)	1.46x	1.31x
Pixel space	1.55x	--

Shuffle Test: What Did the Models Learn?

CLS Token

1. Multi-frame aggregation (order-independent)
Shuffled model (211.9) still far better than copy (620)
Combining all previous frames >> last frame

2. Temporal dynamics matter, just dont know how much -
20% degradation from shuffling (176.9 -> 211.9)

Copy shuffle ratio is 11.2x because random CLS pairs are very different -- but it still works, it's using all frames, not just the last one

Patch Tokens

Shuffled model (1114) = copy baseline (1116)
ALL improvement over copy-last destroyed by shuffling

No benefit of running summary (unlike CLS) -- patches are mostly static, one frame is a sufficient static predictor

DINO patches shuffle ratio (1.46x) behaves like pixel space (1.55x)

100% of model's advantage = temporal order

CLS or Patch Prediction?

CLS

+ Cleaner training signal, no gradient dilution
+ Every frame contributes meaningfully
- Captures semantic shifts, not just motion (next slide)
- Throws away spatial info stage 2 needs

Patches

+ Right inductive bias: spatial + temporal
+ Better aligned with stage 2 forecasting
- Gradient dilution from static patches
- Trails CLS on UCF by 4-7 pts

Fixing patch gradient dilution:

1. Motion-weighted loss -- upweight dynamic patches in L2 loss
2. Predict residuals ($x_{t+1} - x_t$) -- static patches become ~zero, dynamic = entire signal
3. CLS as auxiliary -- keeps gradients flowing when patch loss stagnates on static tokens

Static vs Dynamic Patch Decomposition

Is the patch model learning dynamics or just smoothing?

Split patches into static (bottom 50% motion) and dynamic (top 50%) per video. Compare improvement over copy baseline.

Patch Group	Copy Baseline	Model	Improvement
Dynamic patches	1430.6	1077.0	24.7%
Static patches	746.9	625.9	16.2%

Improvement 1.5x more concentrated on dynamic patches -- model learned dynamics, not just smoothing

Dynamic-only copy shuffle ratio: 1.40x (vs CLS 11.2x)

CLS captures something beyond patch-level motion -- holistic semantic shifts

Next Steps

A) Fix learned-space collapse

JEPA-style h=W(DINO) + SigReg

Currently collapses without CE loss

Try: SigReg regularization

Try: EMA / stop-gradient on target encoder

B) More expressive W_pred

Recurrent state is NOT the bottleneck

Close patch gap: RNN 851 vs transformer 783

Goal: match DINO mean-pool on UCF (88%)

More expressive predictor, not bigger state

Diagnostics to run:

OOD prediction decay curve (SSv2 vs UCF101 -- general dynamics or SSv2-memorized?)

Multi-step autoregressive rollout (extrapolate trajectories or collapse to mean?)

Deva’s distillation

Stage 1-2: learn a temporal DINO feature that encodes frames [t-K:t] with a juicy finite representation (e.g., 700). This can be learned from passive observations of (real-world) video with self-supervised objective (DINO, flow).

My bet: encode(DINO[t-K:t]) => decode

Stage 3: (key bet). By learning the temporal feature above, we can train lightweight policies in IsaacSim (not on pixels but temporal DINO features). This resonates with current best practices that train blind “state-based RL” policies in sim using very small MLPs.